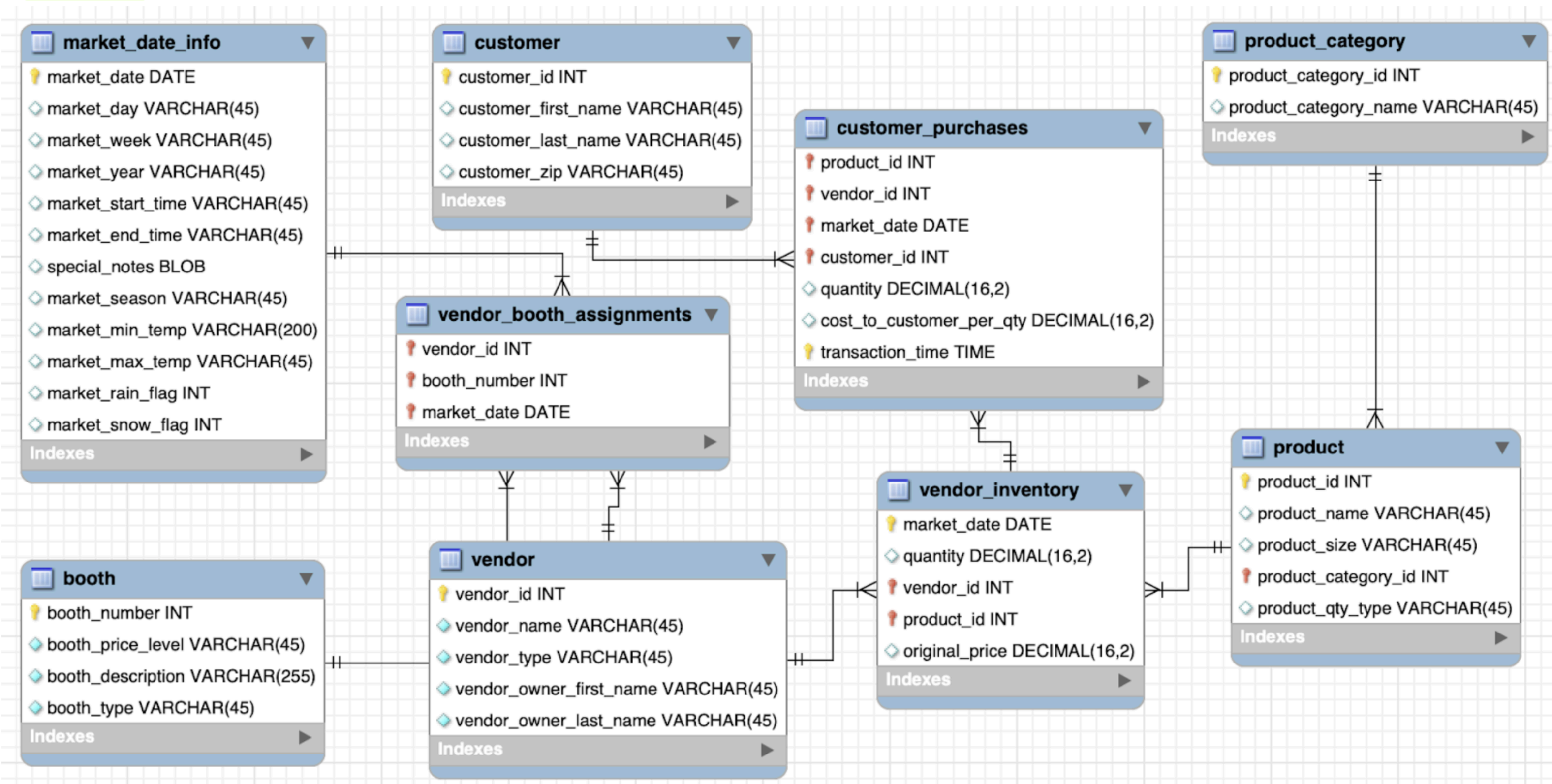
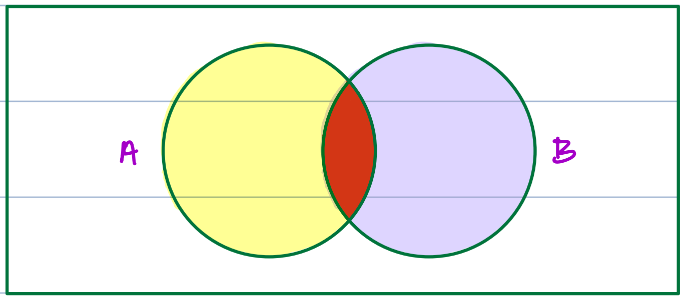

## WhatsApp Link

https://chat.whatsapp.com/EvVhJ555AGW1cJaeKI0USe

## Dataset



## Full Join / Full Outer Join




You don't have implementation of full join in mysql

$$A = \{1,2,3,4,5\} \cup B = \{4,5,6,7,8\} \xrightarrow{\text{union}} \{\underbrace{1,2,3}_{L}, \overbrace{6,7,8}^{R}, \underbrace{4,5}_{\text{common}}\}$$

A

| w | x |
|---|---|
| 1 | a |
| 2 | b |
| 3 | c |
| 4 | d |

B

| y | z |
|---|---|
| 3 | m |
| 4 | n |
| 5 | o |
| 6 | p |

$\xrightarrow[\text{Join}]{\text{Left}}$

| w | x | y | z |
|---|---|---|---|
| 1 | a | null | null |
| 2 | b | null | null |
| 3 | c | 3 | m |
| 4 | d | 4 | n |

A.w = B.y

| w | x |
|---|---|
| 1 | a |
| 2 | b |
| 3 | c |
| 4 | d |

| y | z |
|---|---|
| 3 | m |
| 4 | n |
| 5 | o |
| 6 | p |

$\xrightarrow[\text{Join}]{\text{Right}}$

| w | x | y | z |
|---|---|---|---|
| null | null | 5 | o |
| null | null | 6 | p |
| 3 | c | 3 | m |
| 4 | d | 4 | n |

Union → Vertically Joining results

Left Join

| w | x | y | z |
|---|---|---|---|
| 1 | a | null | null |
| 2 | b | null | null |
| 3 | c | 3 | m |
| 4 | d | 4 | n |
| | | | |

Right Join

| w | x | y | z |
|---|---|---|---|
| null | null | 5 | 0 |
| null | null | 6 | p |
| 3 | c | 3 | m |
| 4 | d | 4 | n |
| | | | |

$\xrightarrow{\text{union}}$

| w | x | y | z |
|---|---|---|---|
| 1 | a | null | null |
| 2 | b | null | null |
| 3 | c | 3 | n |
| 4 | d | 4 | n |
| null | null | 5 | 0 |
| null | null | 6 | p |
| | | | |

Union All → Vertical Join with duplicates in place

| w | x | y | z |
|---|---|---|---|
| 1 | a | null | null |
| 2 | b | null | null |
| 3 | c | 3 | m |
| 4 | d | 4 | n |
| | | | |

| w | x | y | z |
|---|---|---|---|
| null | null | 5 | 0 |
| null | null | 6 | p |
| 3 | c | 3 | m |
| 4 | d | 4 | n |
| | | | |

$\xrightarrow[\text{all}]{\text{union}}$

| w | x | y | z |
|---|---|---|---|
| 1 | a | null | null |
| 2 | b | null | null |
| 3 | c | 3 | m |
| 4 | d | 4 | n |
| null | null | 5 | 0 |
| null | null | 6 | p |
| 3 | c | 3 | m |
| 4 | d | 4 | n |
| | | | |

## Union Rules

1. Every select statement within UNION must have same number of columns
2. The columns must have same | similar datatypes

## Equi and Non-equi Joins

| | |
|---|---|
| Equi Join | on table1.id = table2.id |
| Non-equi Join | on table1.id != table2.id |

| cid | cname | dept_id |
|---|---|---|
| 1 | ankush | 101 |
| 2 | dheeraj | 102 |
| 3 | prajyot | 103 |
| 4 | vicky | 105 |
| 5 | mayank | 101 |

| dept_id | d-name |
|---|---|
| 101 | USA |
| 102 | INDIA |
| 103 | UK |
| 104 | AUS |
| 105 | JAPAN |

**Option 1:** Should we join based on dept_id = dept_id

**Option 2:** Try joining on cust.dept_id != dept.dept_id

Home work for Non Equi Join

| cid | cname | dept_id | dname |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

Corelated Subquery

Basic subquery

| eid | name | d_id |
|---|---|---|
| 1 | A | 101 |
| 2 | B | 101 |
| 3 | C | 102 |
| 4 | D | 103 |

| d_id | d_name | location |
|---|---|---|
| 101 | IT | India |
| 102 | Admin | India |
| 103 | Finance | USA |
| 104 | Accounts | USA |

```
Select eid, name from emp where d_id in (
    select d_id from dept where location = 'India'
);
```

① Run Inner Query, store result in mem

② Run outer query using above result

| eid | name | d_id |
|---|---|---|
| 1 | A | 101 |
| 2 | B | 101 |
| 3 | C | 102 |
| | | |

$\boxed{101, \ 102}$

Corelated SQ - Short circuiting, Huge data

select eid, name, d_id from emp e where exist (
    select * from dept d where e.d_id = d.d_id
    and location = "India"
);

| eid | name | d_id |
|---|---|---|
| 1 | A | 101 |
| 2 | B | 101 |
| 3 | C | 102 |
| | | |

10:10 pm

↓

10: 15 pm

① We will reference some value of outer query inside inner Query

② Best to use in case of huge data

## Aggregation function

① Min ② Max ③ Sum ④ Count ⑤ Avg

| id | name | salary | dept_id |
|---|---|---|---|
| 1 | Dhiraj | 12000 | 101 |
| 2 | Atul | 16000 | 102 |
| 3 | Roshini | 15000 | 103 |
| 4 | chandni | 30000 | 101 |
| 5 | sabari | 8000 | null |

## Count

| Count (*) | count (1) | count (col_n) | count (dist col) |
|---|---|---|---|
| Counts NULL<br>- commonly used<br>- less confusing | Exactly same as count(*) | Counts duplicate & ignores NULL | Does not count duplicates and ignores NULL |

## Min / Max / sum / Avg

| id | name | salary | dept_id | |
|---|---|---|---|---|
| 1 | Dhiraj | 12000 | 101 | 0 |
| 2 | Atul | 16000 ✓ | 102 | 1 |
| 3 | Roshini | 15000 ✓ | 103 | 1 |
| 4 | chandni | 30000 ✓ | 102 | 1 |
| 5 | sabari | 8000 | 101 | 0 |

→ min (salary)

8000

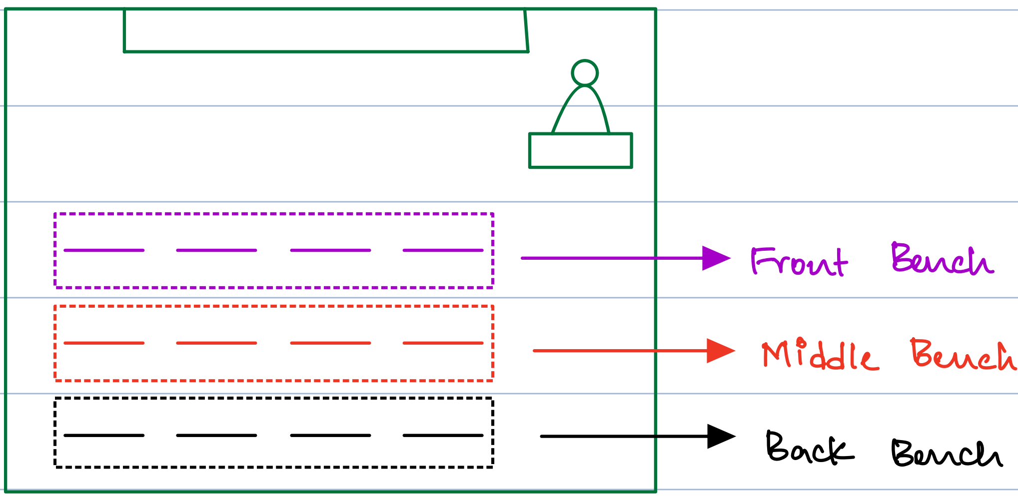
→ sum (salary)

81000

→ sum (salary > 12000)

3

## Group By




## Syntax

Select $col_1$ , $col_2$ from table

where condition

Group by $col_1$ , $col_2$ ;

## Rules

1. Whatever you have in group by columns, make sure they are present in select statement
2. To filter on aggregated column use the having clause

## Syntax vs Execution Order

| Syntax Order | Execution Order |
|---|---|
| Select | From |
| From | where |
| where | group by |
| Group by | aggregated cols |
| Having | having |
| order by | Select |
| limit | order by |
| Offset | offset |
| | limit |

## Having clause

Similar to where clause, having clause is used for filtering results

where → works with columns that are in table

having → works on aggregated columns, which are natively not present in table